

Colorado School of Mines

Data Science

DSCI570OL: Introduction to Machine Learning

DNS Threat Detection

Final Project

Author

Paul Lintzen
paul.lintzen@mines.edu

Instructor

Dr. Zibo Wang
zibowang@mines.edu

Submitted: May 13, 2026

Contents

1	Introduction	2
1.1	Problem Statement and Motivation	2
1.2	Background	2
1.3	Definition of Success	3
2	Technical Details	3
2.1	Datasets	3
2.1.1	Supervised – Public DNS Dataset	3
2.1.2	Unsupervised – Local Network DNS Data	4
2.2	Data Wrangling and Cleaning	5
2.3	Models	9
2.3.1	Stage 1 – Supervised Classifier	9
2.3.2	Stage 2 – Unsupervised Anomaly Detector	9
2.4	Model Training	9
2.4.1	Stage 1 – Supervised Classifier Training	9
2.4.2	Stage 2 – Unsupervised Anomaly Detector Training	11
2.5	Model Performance Results	11
2.5.1	Stage 1 – Supervised Classifier Results	11
2.5.2	Stage 2 – Unsupervised Anomaly Detector Results	13
2.5.3	Stage 3 – Ensemble Results	15
3	Challenges and Conclusion	15
3.1	Reflection on Goals	16
3.2	Future Work	16
3.3	Lessons Learned	16
4	Acknowledgments and References	16
4.1	References	17

1. Introduction

1.1. Problem Statement and Motivation

The core problem addressed in this project is detecting suspicious network activity. For the purposes of this project, I will be working with my home network as a sample but it is applicable to any network. Specifically, rather than looking at the entirety of network traffic which can be very resource-intensive, I will only be looking at DNS logs. To collect these logs I set up Pi-hole, a popular Raspberry Pi program that allows you to use a Raspberry Pi as a local DNS server to filter network traffic and block selected domains. Additionally, it allows configurable logging of all queries along with how they were handled. I am curious if these logs can be used to detect anomalous activity on my home network and can be a lightweight alternative to full network monitoring. Anyone who cares about their online security and particularly the security of their local network may be interested in this product as it can flag suspicious activity that might otherwise go unnoticed. While most homeowners won't be super interested in this, in environments like data centers and industrial control system where network security is critical a lightweight network monitor could be an extremely useful tool. There are many current network solutions but most focus on a wider look at full network activity as this provides much more information as opposed to just DNS logs. I hope that my project will be able to create a lightweight alternative to these approaches that can be applied anywhere and still perform well.

1.2. Background

As a quick introduction, DNS stands for Domain Name System and it can be thought of like a phone book for the internet. When you visit a website your device sends that URL to a DNS server that translates it into an IP address that your traffic can then be routed to. So in the phone book analogy, when you tell your computer you want to contact google.com, it checks that name in the phone book to find the correct number and routes to that IP address.

Pi-hole is just a DNS server that you can run locally. This allows you to not just look up domain names' IP addresses but also filter them. So when a device on your network makes a DNS request Pi-hole checks it against a list of known ad or tracking domains and blocks it if found. This allows you to block ads network wide without needing device level ad blockers. One limitation of this is that it can only block known ads that are already on the block list. So if for example a new ad domain pops up or another malicious domain exists outside of the known block list it will not be blocked. For my setup, I used 4 publicly available block lists

as a base, totally 603,095 unique blocked domains. These lists are Steven Black's host list (default), OISD big block list, the URLHaus abuse list, and HaGeZi's Pro DNS Blocklist.

1.3. Definition of Success

I would classify this project a success if I am able to achieve the following.

For supervised model:

- $> 95\%$ f1/auroc/auprc on the test set

For the unsupervised model:

- Realistic anomaly rate between 1% and 3%
- Top flagged events make sense after manual inspection

These would be meaningful results because in a real world application we want to ensure the model catches as many malicious queries as possible without flagging benign queries as well. Around 95% f1 indicates a solid balance between recall and precision. And for our unsupervised portion since this is my home network I expect very low over anomaly rate and I can hopefully identify from visual inspection if queries are actually unusual.

2. Technical Details

This system uses a two-stage ensemble approach. The first stage is a supervised classifier trained on a labeled public DNS dataset to detect known malicious query patterns. The second stage is an unsupervised anomaly detector trained on local network DNS traffic to flag queries that deviate from normal behavior. The main idea of this approach is that the supervised model catches known threats, while the unsupervised model finds novel or internal anomalies.

2.1. Datasets

2.1.1. Supervised – Public DNS Dataset

- **Source:** [DGTA-BENCH - Domain Generation and Tunneling Algorithms for Benchmark](#)
- **Size:** 1,657,234 labeled domain names
- **Input feature:** domain: raw text url
- **Output / Label:** class: label as one of 55 classes (4 DTA, 50 DGA, 1 legitimate)
- **Example:**

Domain	Class
jpvtpszhhqqvz.com	bedep
0dev.null.googlepages.com	legit
q+Z78XwJBA.hidemyself.org	dns2tcp

Table 1: Example labeled records from the public DNS dataset.

2.1.2. Unsupervised – Local Network DNS Data

- **Source:** DNS query logs collected from Pi-hole
- **Size:** 4,503,855 queries from January 25, 2026 (12:00AM) to April 25, 2026 (11:59PM)
- **Input features:**

Label	Type	Empty Okay	Content
id	integer	No	Autoincrement ID for the table, only used by SQLite3, not by <i>FTLDNS</i>
timestamp	integer	No	Unix timestamp when this query arrived at <i>FTLDNS</i> (used as index)
type	integer	No	Type of this query
status	integer	No	How was this query handled by <i>FTLDNS</i> ?
domain	text	No	Requested domain
client	text	No	Requesting client (IP address)
forward	text	Yes	Forward destination used for this query (only set if <code>status == 2</code>)
additional_info	blob	Yes	Data-dependent content, see below
reply_type	integer	Yes	Type of the reply for this query
reply_time	real	Yes	Seconds it took until the final reply was received
dnssec	integer	Yes	Type of the DNSSEC status for this query
regex_id	integer	Yes	ID of the regex filter that matched this query (only set if blocked by a regex filter)

Table 2: Query table schema

For more information on input features, see the [Pi-hole documentation](#).

- **Example:**

time	domain	type	status	reply_type	reply_time	forward
2026-01-25 06:00:04	v10.events.data.microsoft.com	1	2	3	0.006923	1.1.1.1#53
2026-01-25 06:00:15	login.live.com	1	3	3	0.000089	NaN
2026-02-28 02:21:37	api.steampowered.com	16	3	1	0.000524	NaN
2026-02-20 23:25:41	elearning.mines.edu	16	3	3	0.000085	NaN

Table 3: Example unlabeled records from my local data.

2.2. Data Wrangling and Cleaning

Public dataset:

- Check for missing values, duplicate records etc.
- Map 55 unique classes to be one of (0) benign or (1) malicious

Local network data:

- Cut data to cover 1/25 12:00 AM to 4/25 11:59 PM (13 complete weeks of data)
- Filter data to focus one device on the network (my desktop PC) - 103,601 total queries
- Create second dataset filtered to just doorbell queries
- Extract date and time from time column and convert UTC to MST time
- Drop rows with mis-formatted domains
- Drop forward column due to 590,527 missing values
- Fill missing reply times with median
- Map numeric type, status, reply_time columns to corresponding text value for easier interpretation

For some more context before getting into the actual models we can examine some patterns in my local data.

Desktop:

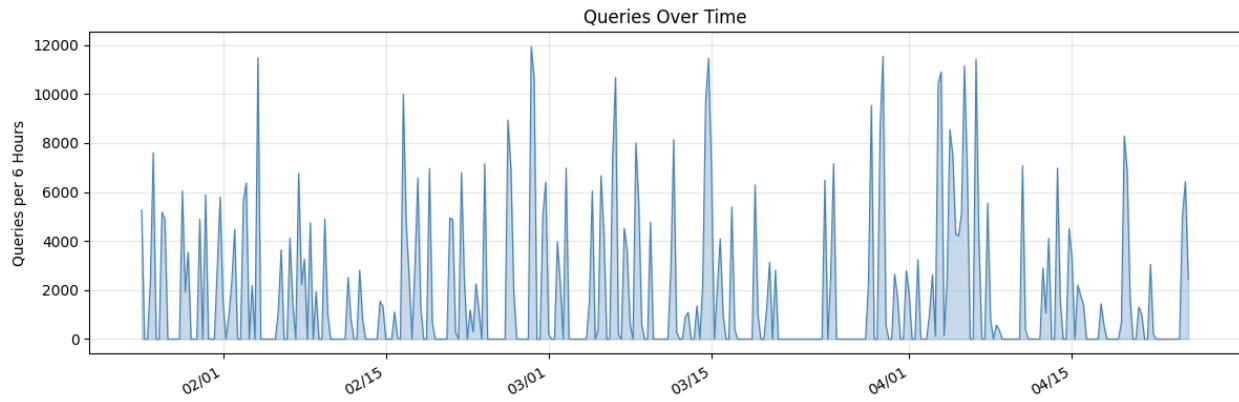


Figure 1: Queries over Time for Desktop PC

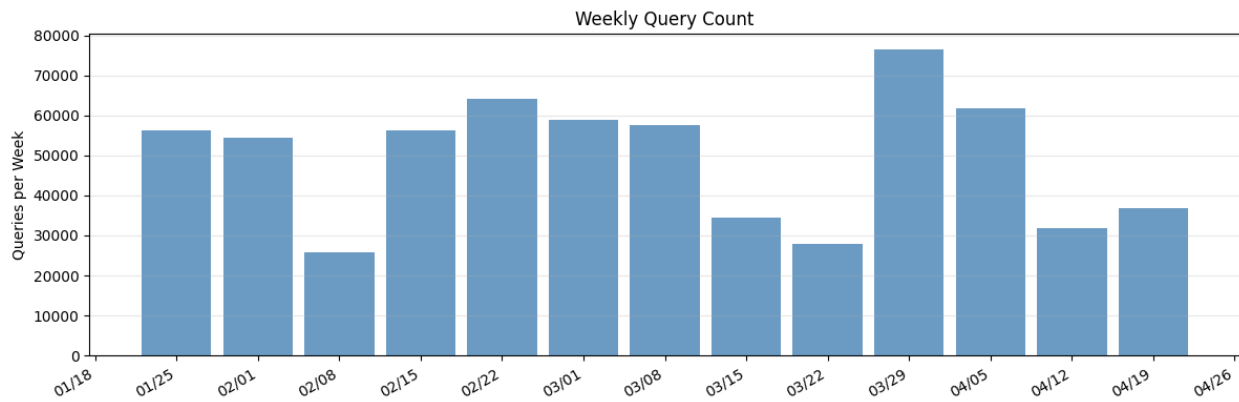


Figure 2: Aggregated weekly queries for Desktop PC

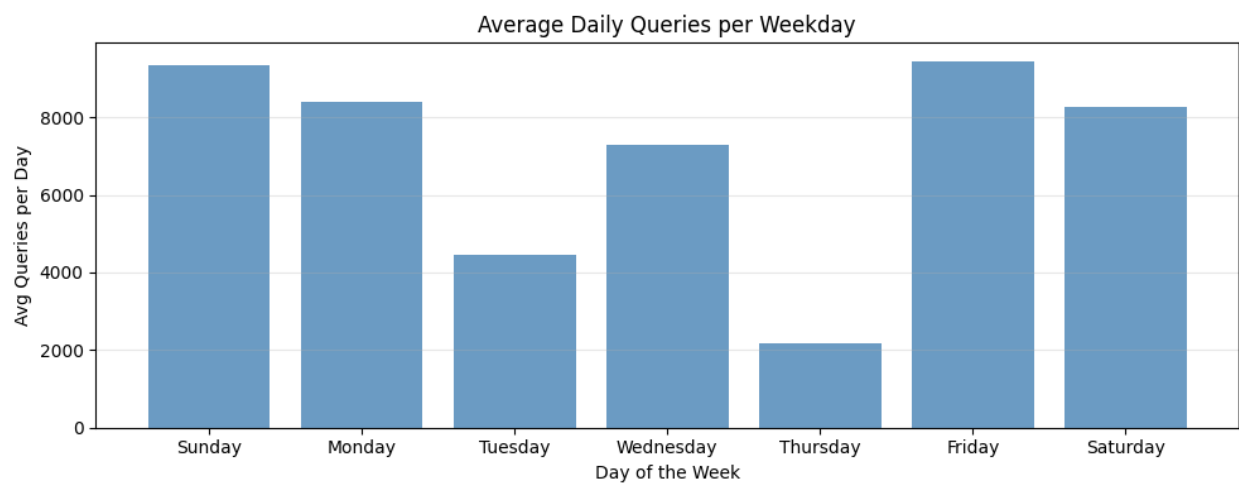


Figure 3: Queries per weekday for Desktop PC

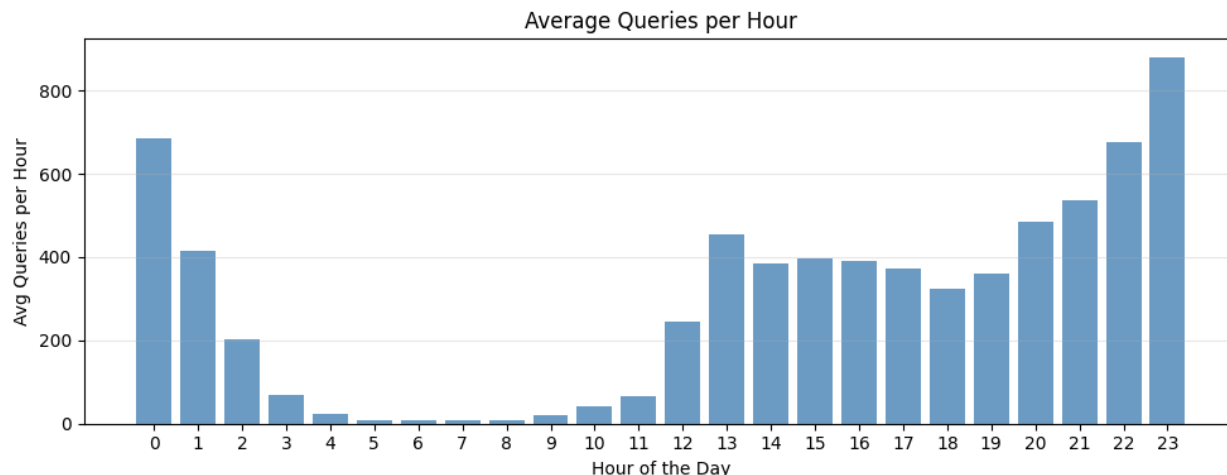


Figure 4: Queries per hour for Desktop PC

From Figure 1 we can see periods of zero activity where the desktop was powered off, and spikes of activity where usage was high. It shows aggregated 6 hours chunks over that 13 week period. To get a better idea of weekly usage Figure 2 shows aggregate weekly queries. We can clearly see some weeks of low activity such as the week of 2/8 and 3/22. The week of March 22nd was spring break and I was traveling and the week of February 8th I had friends visiting so the reduced usage makes sense. Next, to get a breakdown of per-weekday activity Figure 3 displays average queries by weekday. We see weekends have more activity and Tuesday's and Thursday's have the least. This again makes sense as Tuesdays and Thursdays are my heaviest class days so I am home much less and as a consequence my computer is on less. Fridays and weekends see the most activity as I'm home the most. It is great to clearly see this trends in the data as it gives me confidence that my logs are accurate and truly capture my daily usage. A final useful visual is average queries by hour of day. This will just be a sanity check to see if the times of most activities make sense. From Figure 5, we can see a clear pattern of nearly zero activity from 4-10am as I am consistently asleep at this time. We also see spikes in the evening hours as this is usually when I return home. These visuals give us a good idea of the baseline activity that our unsupervised model will be trained on.

Doorbell

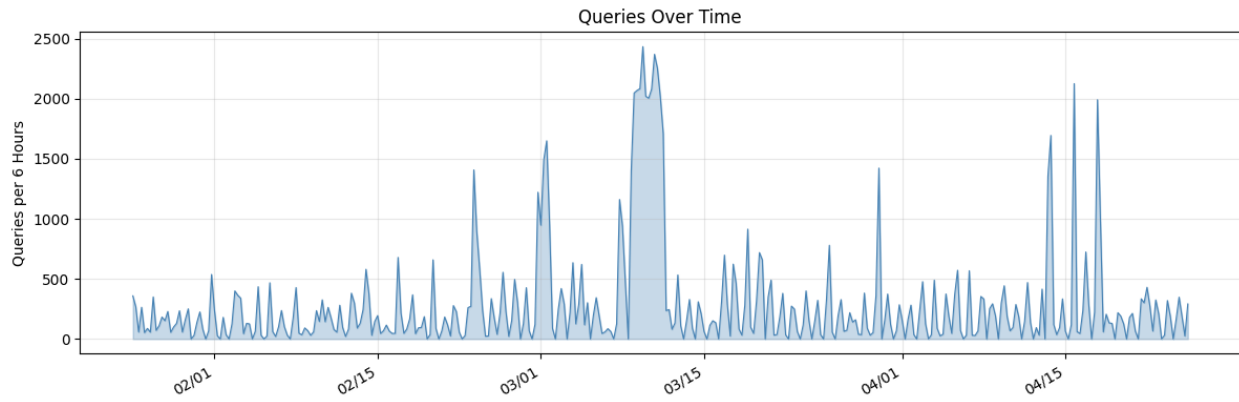


Figure 5: Queries over Time for Doorbell

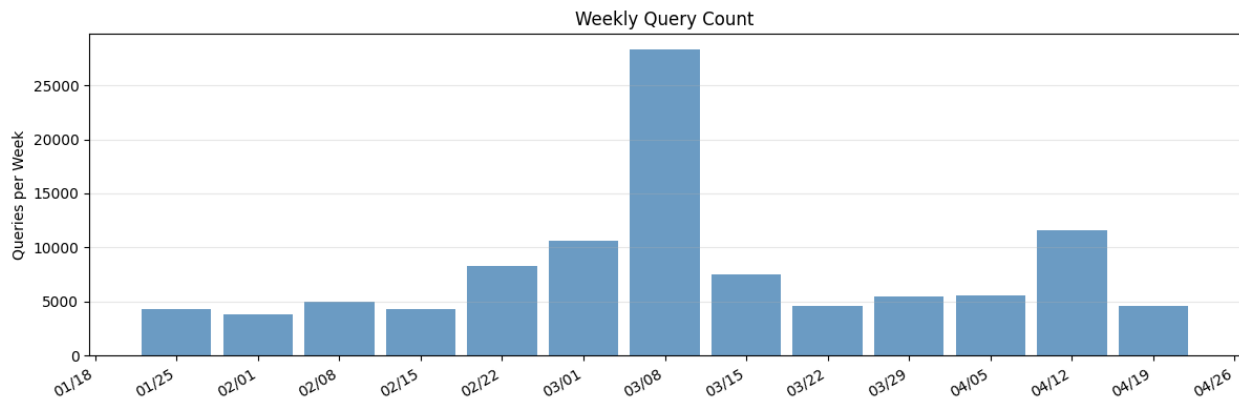


Figure 6: Aggregated weekly queries for Doorbell

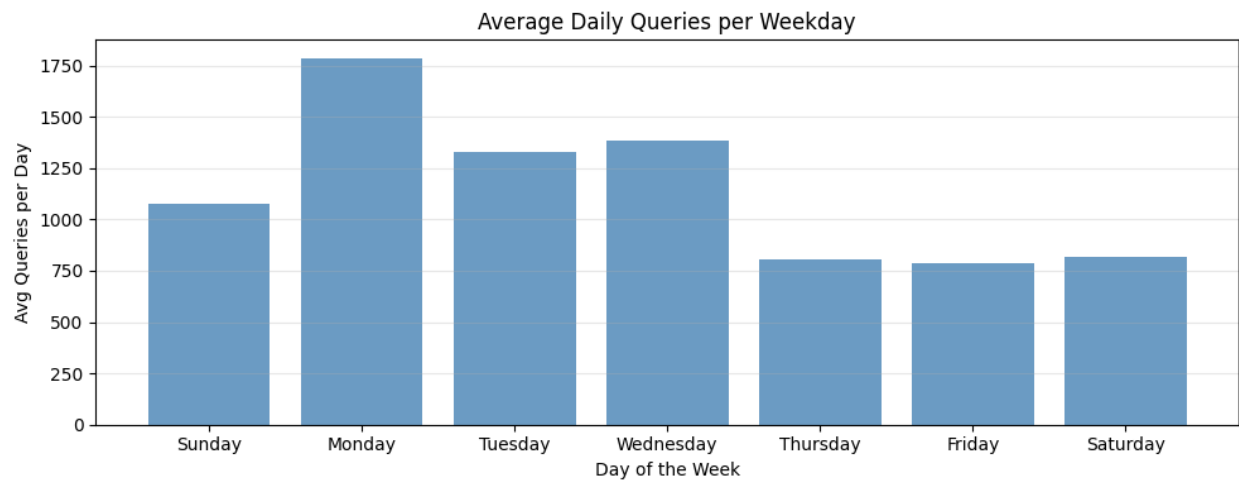


Figure 7: Queries per weekday for Doorbell

Our doorbell data also reveals some very interesting trends. Unlike the PC this device was

never powered off so we see more consistent queries across the entire time period in Figure 5. We also see a massive spike in activity around mid march. This is even more apparent in Figure 6. My family visited during this time explaining the massive increase in activity. The per-weekday data in Figure 7 is also interesting here as we can see a different trend than in the PC data.

2.3. Models

2.3.1. Stage 1 – Supervised Classifier

- **Model type:** XGBoost
- **Configuration:** `n_estimators = 300`, `learning_rate = 0.05`, `max_depth = 8`, `subsample = 0.8`, `colsample_bytree = 0.7`, `random_state = 42`
- **Why this model:** XGBoost handle tabular data extremely well and can compete with neural networks' performance while being much faster

2.3.2. Stage 2 – Unsupervised Anomaly Detector

- **Model type:** Isolation Forest
- **Architecture / Configuration:** `n_estimators=300`, `contamination=0.01`, `random_state=42`
- **Anomaly threshold:** Set at 1% as a reasonable estimate since true contamination on my home network is unknown.
- **Why this model:** Efficient unsupervised model that works well with high-dimensional tabular data. Also naturally produces continuous anomaly scores that can be used later in ensemble.

2.4. Model Training

2.4.1. Stage 1 – Supervised Classifier Training

Feature learning:

The most basic feature I started with was just the length of the domain. I then continuously added more until I performance stopped improving. All of the features can be seen below in Table 4.

Feature	Description
len	length of the domain
tld_freq	frequency of top level domain in training dataset (.net, .com, .xyz, etc)
subdomain_depth	number of '.' in the domain string
entropy	measure of randomness in the domain string
num_ratio	ratio of numbers to all other symbols
consonant_ratio	ratio of consonants to all other symbols
longest_token	longest string of characters between '.' symbols

Table 4: Engineered features for labeled dataset

To understand how much our model actually relied on each of these features we can look at the feature importances in 8. This shows our model relies heavily on subdomain_depth.

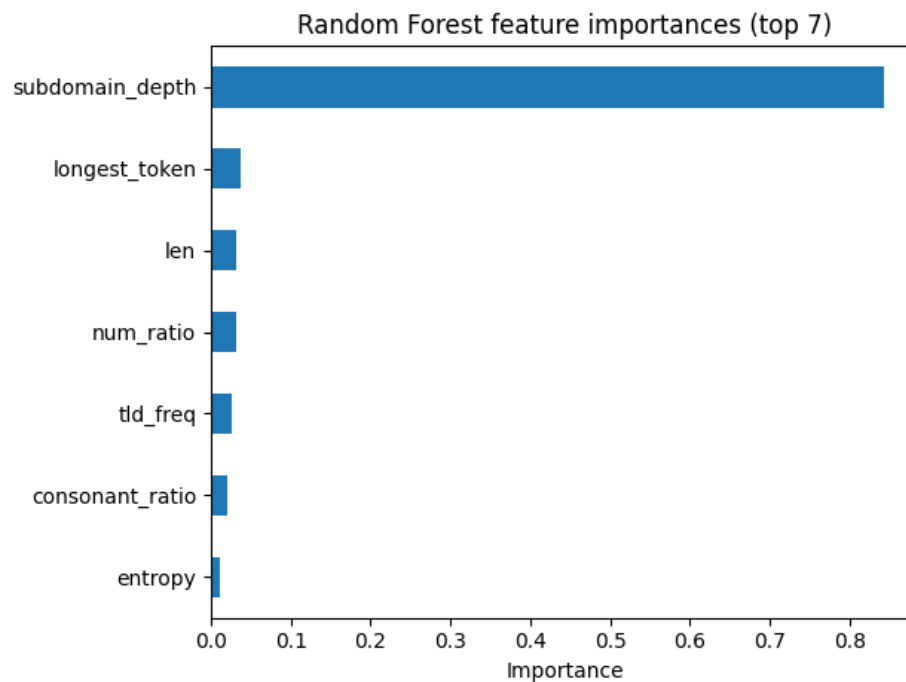


Figure 8: Feature importances for final model.

Data splitting:

- **Split:** 70% train / 15% validation / 15% test
- **Method:** Train model on 70%, tune hyperparameters on 15%, and test on final 15%.
- **Cross-validation:** 5-fold CV used for hyperparameter tuning

2.4.2. Stage 2 – Unsupervised Anomaly Detector Training

Feature learning:

As mentioned previously the first simple steps were just converting date and time from UTC to MST and filling in missing values. Next, I mapped the numerical indicators for type, status, and reply_type to their corresponding string labels. Then I engineered the features seen in Table 5. For more information on the structure of the type, status, and reply_type columns see the [Pi-hole documentation](#).

Feature	Description
time_sin/time_cos	cyclical encoding of hour of the day
dow_sin/dow_cos	cyclical encoding of day of the week
is_weekend	boolean flag to provide further differentiate weekends
reply_time	raw reply_time value from logs
status_category	one-hot encoded status (allowed, blocked, etc)
query_type	one-hot encoded type (HTTPS, A, AAAA, etc)
reply_type	one-hot encoded reply type of the server (NXDOMAIN, CNAME, BLOB)

Table 5: Engineered features for unlabeled local dataset

Data splitting: Because there were no labels for this data, I did not split it. 100% of the data was analyzed to establish baselines and identify anomalies.

2.5. Model Performance Results

2.5.1. Stage 1 – Supervised Classifier Results

Relevant classification metrics on the held-out test set.

Model / Version	Accuracy	Precision	Recall	F1	AUC-ROC
Baseline	0.696	0.662	0.257	0.370	0.690
v2	0.797	0.758	0.612	0.677	0.849
v3	0.970	0.971	0.940	0.955	0.992
v4	0.973	0.972	0.949	0.960	0.995
v5	0.980	0.973	0.968	0.970	0.997
Final Model	0.986	0.975	0.986	0.980	0.998

Table 6: Supervised model improvement over iterations.

Iteration history:

Version	Change Made
Baseline	Initial model with only the domain length
v2	Added top level domain frequency
v3	Added subdomain depth
v4	Added entropy score
v5	Added number and consonant ratios
Final Model	Added longest token

Table 7: Supervised model changes over iterations.

ROC curve:

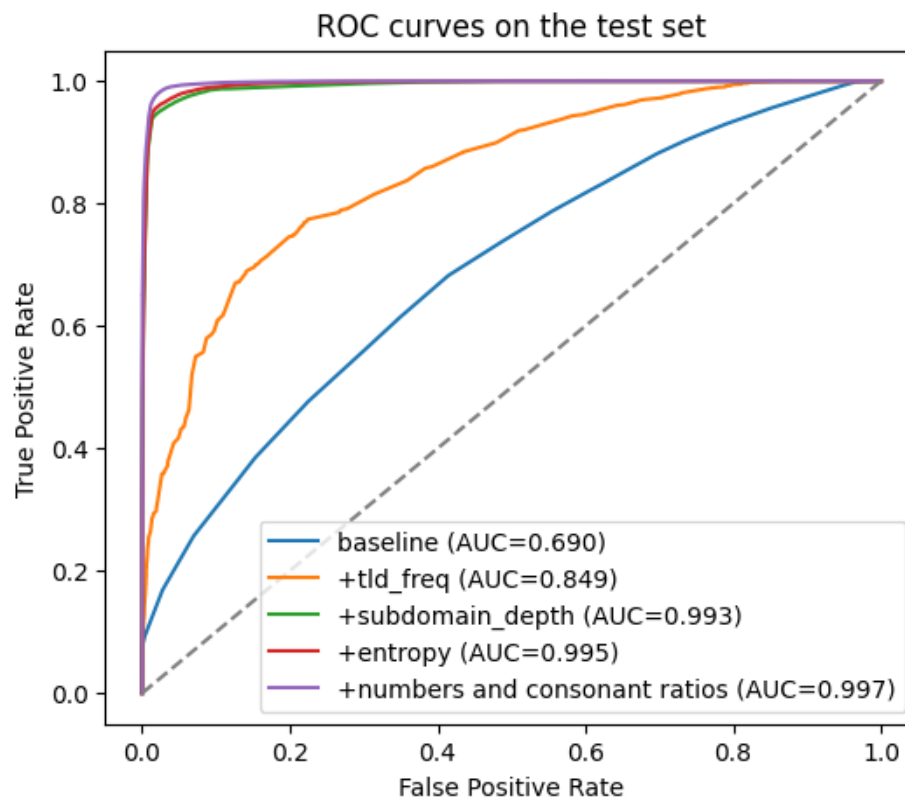


Figure 9: ROC curve across training iterations for the supervised classifier.

We can see that our model performance increased drastically as we added more features initially but then slowed down once the most important ones were in. Our final model performance is above the 95% f1 score goal we set so this overall I am happy with how my final model performed.

2.5.2. Stage 2 – Unsupervised Anomaly Detector Results

- **Anomaly score distribution:** The anomaly score distribution (see in Figure 10) was heavily left tailed. This is expected as the majority of our queries should not be anomalous.

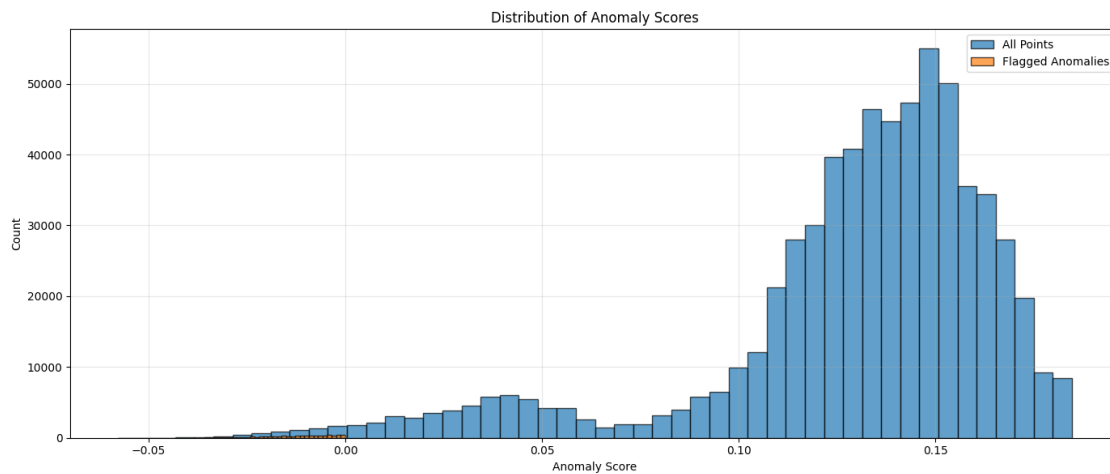


Figure 10: Total anomaly score distribution.

To get a better picture of the queries we classified as anomalous with our 1% threshold we can zoom in. In Figure 11, we can see that only a the most extreme anomaly scores were truly flagged as anomalous.

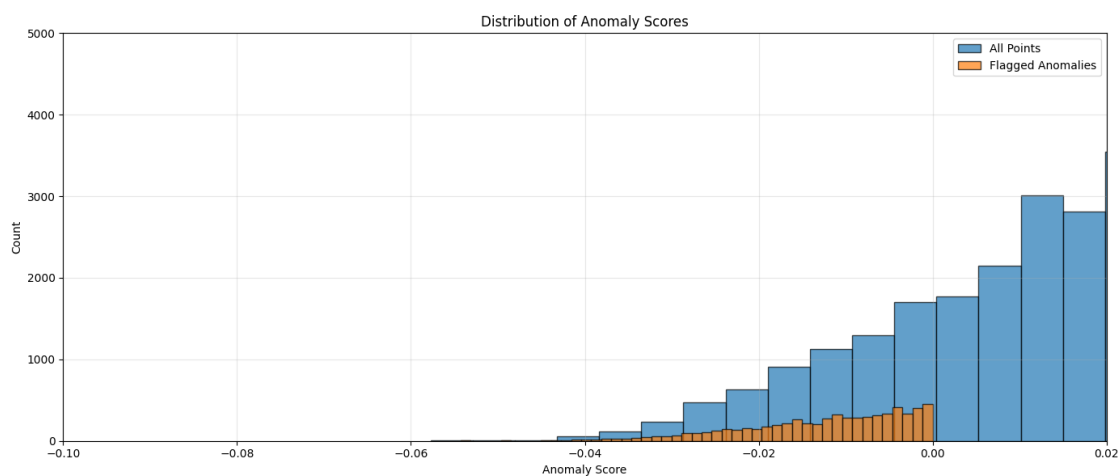


Figure 11: Zoomed anomaly score distribution.

Another way we can analyze this is looking at one key metric we trained on which was time of day in Figure 12.

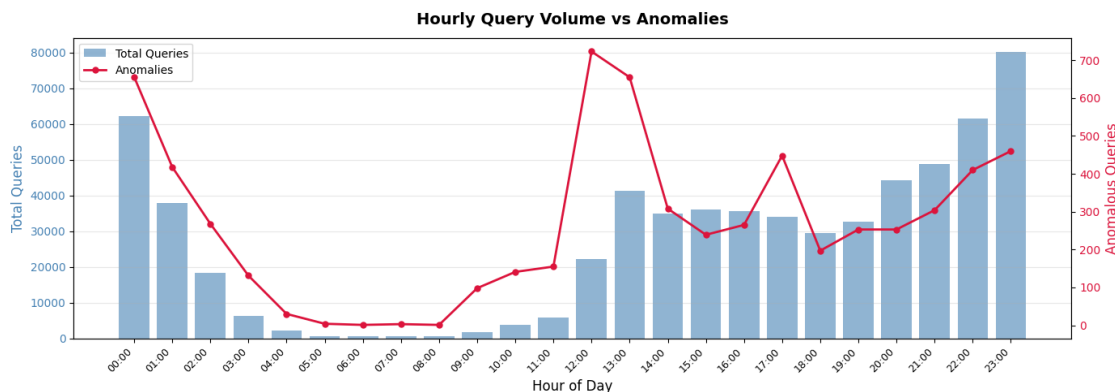


Figure 12: Anomalies by time of day.

Here we can see a clear peak in anomaly rate compared to total queries around mid day. This is very interesting behavior and my tentative explanation would be related to windows update services. Because I generally turned my computer on later in the afternoon/evening, the days where I did turn it on around 12 saw strange queries as the services requested during Windows boot up likely look nothing like normal traffic.

- **Flagged events:** At 1% contamination, 6421 queries were flagged as anomalous. To get an idea of the types of queries that are being flagged we can look at the top 20 with the worst anomaly score in Figure 13.

domain	time	day_of_week	TYPE	REPLY	STATUS	anomaly_score
www.bestlaptop.deals	2026-03-25 10:54:43	Wednesday	HTTPS	CNAME	allowed_forwarded	-0.057753
mines.evaluationkit.com	2026-04-04 09:37:16	Saturday	HTTPS	CNAME	allowed_forwarded	-0.057744
store.steampowered.com	2026-04-04 09:34:07	Saturday	A	validIP	allowed_forwarded	-0.054171
gaminglaptop.deals	2026-04-04 09:35:00	Saturday	A	validIP	allowed_forwarded	-0.054171
95-216-39-237.top	2026-04-04 09:35:24	Saturday	HTTPS	NODATA	allowed_forwarded	-0.054088
title.mgt.xboxlive.com	2026-03-25 10:42:08	Wednesday	A	CNAME	allowed_forwarded	-0.049824
stun1.l.google.com	2026-03-01 23:55:05	Sunday	AAAA	validIP	allowed_forwarded	-0.049665
random.org	2026-03-05 12:58:21	Thursday	A	validIP	allowed_forwarded	-0.049648
helpcenter.mines.edu	2026-03-25 10:53:49	Wednesday	HTTPS	CNAME	allowed_forwarded	-0.048926
deviceinfo.capitalone.com	2026-03-25 11:13:07	Wednesday	HTTPS	CNAME	allowed_forwarded	-0.048797
wizard.samsungsemi.com	2026-03-14 10:34:14	Saturday	A	CNAME	allowed_forwarded	-0.048387
how-to-pc.info	2026-04-04 09:35:14	Saturday	A	validIP	allowed_forwarded	-0.048066
stun1.l.google.com	2026-03-01 23:55:05	Sunday	AAAA	validIP	allowed_forwarded	-0.045265
d.basemaps.cartocdn.com	2026-03-25 11:16:28	Wednesday	A	CNAME	allowed_forwarded	-0.044975
visit-nagano.alpico.co.jp	2026-02-23 12:49:02	Monday	HTTPS	CNAME	allowed_forwarded	-0.044760
afd-k-acdc-direct.office.com	2026-03-05 11:57:09	Thursday	HTTPS	CNAME	allowed_forwarded	-0.044539
stats.steaminventoryhelper.com	2026-03-25 10:43:21	Wednesday	A	validIP	allowed_forwarded	-0.044275
api.steaminventoryhelper.com	2026-03-25 10:43:21	Wednesday	A	validIP	allowed_forwarded	-0.044275
pintheufo.h5games.usercontent.goog	2026-02-04 03:05:27	Wednesday	A	validIP	allowed_forwarded	-0.044244
www.brandcolorcode.com	2026-02-04 03:02:08	Wednesday	A	CNAME	allowed_forwarded	-0.044211

Figure 13: Unsupervised highest anomaly events.

Looking at these results we see that some do truly seem strange (like 95-216-39-237.top at 9:35 AM on a Saturday) while others (like helpcenter.mines.edu at 10:53 AM on a Wednesday) do not. This means that what our model is flagging as anomalies likely

needs more tuning to ensure we are avoiding flagging normal queries. Importantly, the unsupervised model did not see any information about the domain as that portion was limited to the supervised classifier. So the fact that many of these queries do have strange domain names means that our model is successfully finding behavioral anomalies that come from strange domains.

2.5.3. Stage 3 – Ensemble Results

Finally to create my ensemble score I used a very simple approach. I first used a MinMax scaler to get both my anomaly and supervised scores to an equal scale. Then I defined:

$$\text{combined_score} = 0.5 \times \text{sup_score_norm} + 0.5 \times \text{anomaly_score_norm}$$

The resulting top 20 queries are shown in Figure 14.

	domain	sup_prediction	is_anomaly	sup_score_norm	anomaly_score_norm	combined_score	STATUS
493455	how-to-pc.info	1	-1	0.948066	1.000000	0.974033	allowed_forwarded
493461	95-216-39-237.top	1	-1	1.000000	0.930244	0.965122	allowed_forwarded
450343	browserbench.org	1	-1	0.990695	0.889226	0.939961	allowed_forwarded
45185	shaderlabs.org	1	-1	0.952684	0.912772	0.932728	allowed_forwarded
502244	prismlauncher.org	1	-1	0.974572	0.885773	0.930172	allowed_forwarded
314268	ntvstream.cx	1	-1	0.990397	0.868985	0.929691	allowed_forwarded
227405	trackercdn.com	1	-1	0.985137	0.865098	0.925118	allowed_forwarded
20011	d1mybsg2h91vjj.cloudfront.net	1	-1	0.964996	0.879509	0.922252	allowed_forwarded
561407	cloudflare.com	1	-1	0.938390	0.905737	0.922064	allowed_forwarded
502248	opencollective.com	1	-1	0.936992	0.903685	0.920339	allowed_forwarded
314330	scdnmain.net	1	-1	0.965426	0.874366	0.919896	allowed_forwarded
261312	meltonfoundation.org	1	-1	0.990876	0.847900	0.919388	allowed_forwarded
75956	byronsharman.com	1	-1	0.964951	0.868896	0.916923	allowed_forwarded
13206	adoptium.net	1	-1	0.932743	0.898208	0.915476	allowed_forwarded
341043	winterparkskirental.com	1	-1	0.972202	0.848412	0.910307	allowed_forwarded
260369	bayareavolleyball.com	1	-1	0.976416	0.841280	0.908848	allowed_forwarded
260473	wisepops.net	1	-1	0.965688	0.851952	0.908820	allowed_forwarded
202098	portersfiveforce.com	1	-1	0.947652	0.859700	0.903676	allowed_forwarded
373498	dmxtdltgndcco.site	1	-1	0.997731	0.795795	0.896763	allowed_forwarded
505866	kurapixel.com	1	-1	0.890049	0.901046	0.895548	allowed_forwarded

Figure 14: Ensemble highest flagged events.

When compared to the queries flagged by our unsupervised model along, we now see some more strange domains mixed in like dmxtdltgndcco.site and ntvstream.cx. However, we also see some very normal domains like cloudflare.com. Interestingly, we also see lots of smaller websites or local companies like Winter Park, Bay Area, Byron Sharman, etc. I think because our supervised model is trained on a public dataset these are being flagged strange despite being completely benign. Overall, it is clear we need more tuning before deploying this model in a real world scenario as we want to make sure we aren't constantly finding normal domains to be malicious.

3. Challenges and Conclusion

3.1. Reflection on Goals

Overall, I am happy with my results. I was able to beat my goal of 95% f1 accuracy on the test set and both my anomaly detection and ensemble method did seem to find some truly out of the ordinary queries. However, it is clear that additional tuning is needed since I am consistently still flagging benign domains as well and the supervised model really doesn't perform great with domains that are more local to an area since it was trained on a public dataset of some very specific domains. I think I would try to expand my training set for the supervised model to include many more varied domains and perhaps add a dictionary of comparison steps that gives the model an indication of if a domain uses real words even if they aren't super common. Additionally, I would try to combat the tld frequency issue discussed in my video presentation since my model uses the training tld frequency even when applied to my local dataset. This is necessary to allow the model to work correctly as it was trained on that underlying frequency assumptions. However, my local traffic likely has a completely different frequency distribution so some other solutions should be tested. Additionally, the true contamination of my home network is unknown and may even be 0, so the assumed 1% may be throwing off my anomaly classification.

3.2. Future Work

My next steps would be going back and really analyzing each feature to understand what is going on under the hood in each model. Then I would also want to analyze the differences in my local dataset with the labeled training dataset to try to adjust for those ahead of time. Once I get the classifications to a point where they are truly reliable, I would also integrate with Pi-hole's FTL API to set up live monitoring on new queries. I think this could be extremely valuable on top of the current block list system. This is discussed in slightly more depth in my presentation as well.

3.3. Lessons Learned

Overall, I learned a lot about how supervised and unsupervised models can be used together and the limitations of both. I think the ensemble did overall do a better job flagging strange queries though more tuning is definitely required.

4. Acknowledgments and References

4.1. References

- [1] <https://docs.pi-hole.net/database/query-database/>
- [2] <https://data.mendeley.com/datasets/2wzf9bz7xr/1>
- [3] https://xgboost.readthedocs.io/en/release_3.2.0/
- [4] https://xgboost.readthedocs.io/en/release_3.2.0/
- [4] <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>